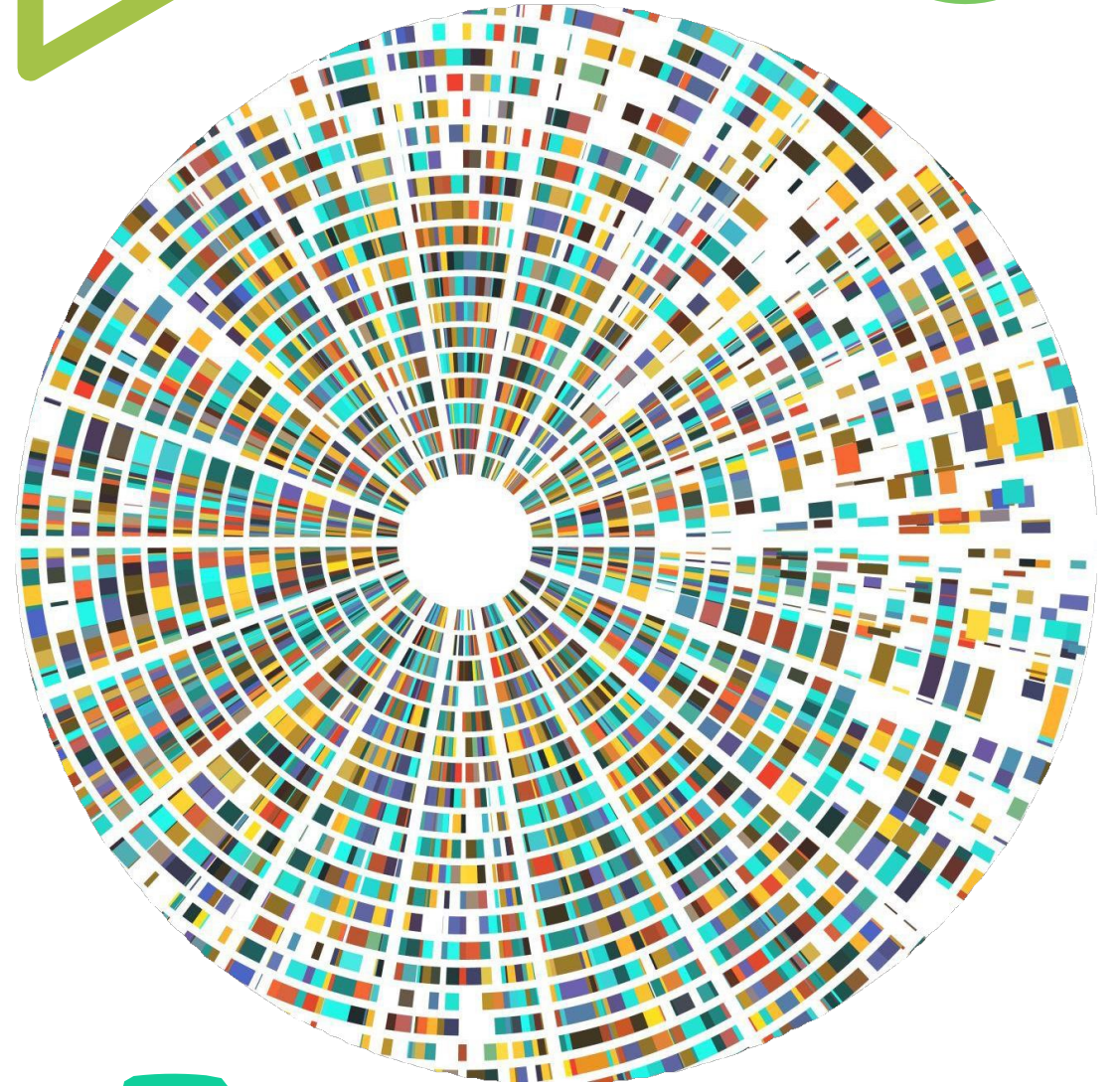




UT-2

Elementos Básicos

1º DAM/DAW
(2024-2025)



Programa principal en java

```
1 public class HolaMundo {
2     public static void main(String[] args) throws Exception {
3         System.out.println("Hello, World!");
4     }
5 }
6
```

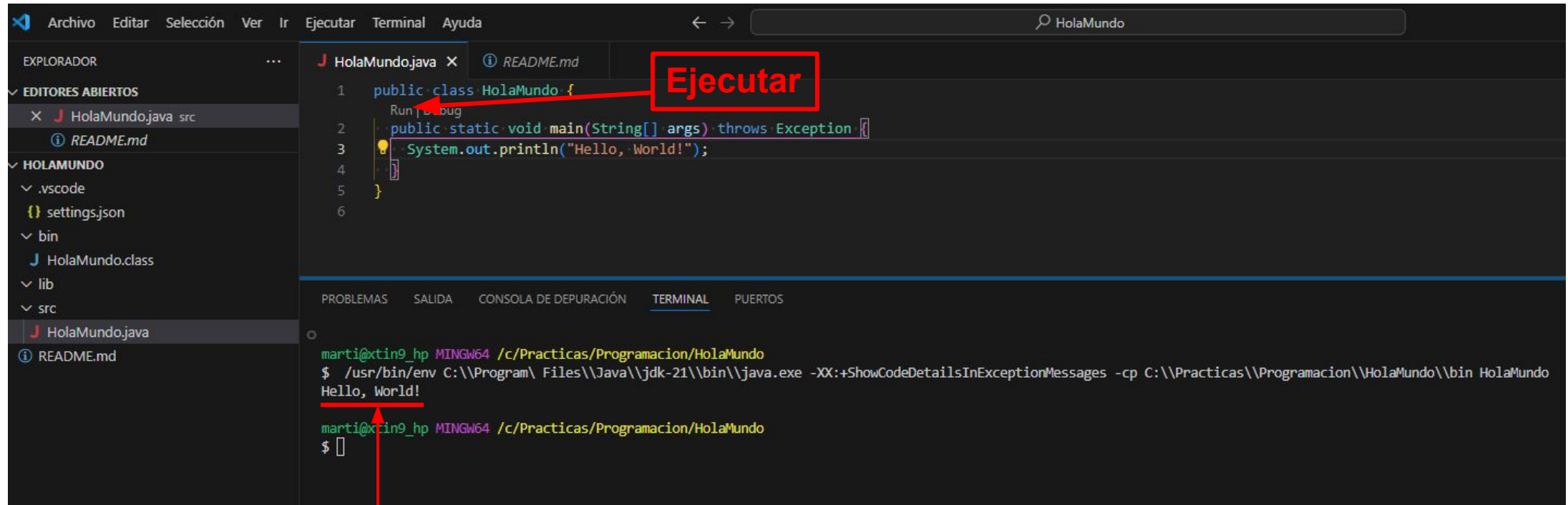
Bytecode

Código Fuente



HolaMundo.java

Programa principal en java



The screenshot shows the Visual Studio Code interface. The Explorer on the left shows the project structure for 'HOLAMUNDO'. The Editor in the center shows the file 'HolaMundo.java' with the following code:

```
1 public class HolaMundo {  
2     public static void main(String[] args) throws Exception {  
3         System.out.println("Hello, World!");  
4     }  
5 }  
6
```

A red box labeled 'Ejecutar' (Execute) is positioned over the 'Run' button in the editor. The Terminal at the bottom shows the command to compile and run the program:

```
marti@xtin9_hp MINGW64 /c/Practicas/Programacion/HolaMundo  
$ /usr/bin/env C:\\Program Files\\Java\\jdk-21\\bin\\java.exe -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Practicas\\Programacion\\HolaMundo\\bin HolaMundo  
Hello, World!  
marti@xtin9_hp MINGW64 /c/Practicas/Programacion/HolaMundo  
$
```

Salida Ejecución Programa

Palabras reservadas

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

true
false
null

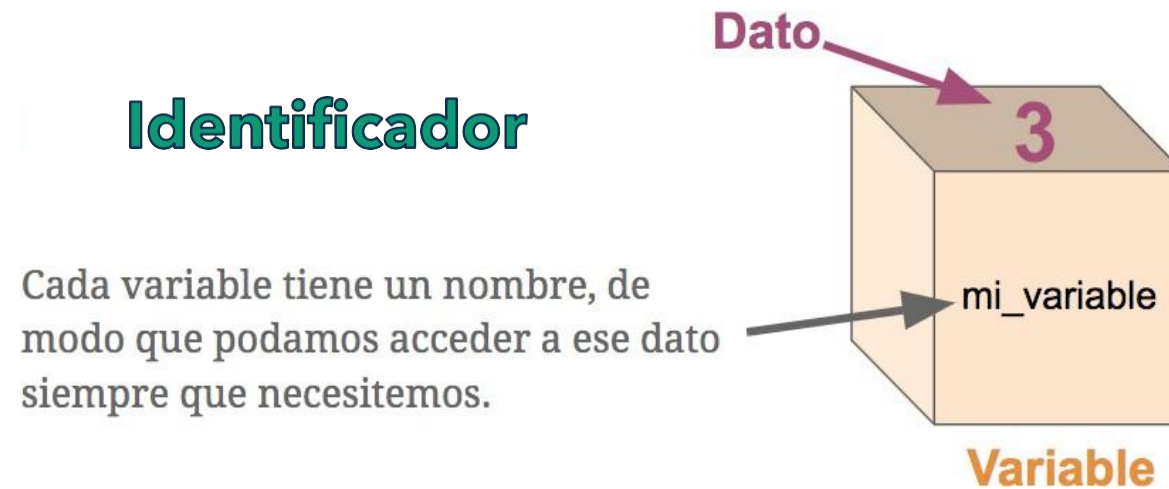
Comentarios

Los comentarios permiten añadir texto al programa que no será procesado por el compilador, por lo que no forma parte del programa. Se utiliza para que el programador pueda añadir notas al código que le permitan dejar explicaciones de algoritmos complicados o de partes del código que deban ser interpretadas en el futuro por él mismo u otro programador.

```
/*  
Este es un comentario que  
ocupa varias líneas  
*/  
  
// Este comentario ocupa solo una línea  
System.out.println("Hello world!"); // comentario al final de la línea
```

Variables

Una variable es una porción de memoria a la que se le asigna un nombre (identificador) y es capaz de almacenar un valor que podrá ser utilizado a lo largo del programa (siempre dependiendo de los modificadores y el contexto del mismo). Además, su valor puede ser modificado en cualquier momento para que pueda ser incrementada, decrementado o simplemente reutilizada.



- Comienza por **letra** (minúscula), carácter (`_`) o carácter (`$`)
- Continúa con **letras**, **dígitos**, carácter (`_`) o carácter (`$`)
- Java es Case Sensitive (**distingue entre mayúsculas y minúsculas**)
- **No tiene longitud máxima**

```
edad
maxValor
notaMediaTrimestre
_usuario
$departamento
```

Variables - Tipos de datos

Tipo	Tamaño	Valor mínimo	Valor máximo
byte	1 byte	-128	127
short	2 bytes	-32768	32767
int	4 bytes	-2147483648	2147483647
long	8 bytes	-9223372036854775808	9223372036854775807
float	4 bytes	-3.402823e38	3.402823e38
double	8 bytes	-1.79769313486232e308	1.79769313486232e308
char	2 bytes		
boolean	1 byte	false	true

En Java, cuando se quiere hacer uso de una variable, se debe escribir el tipo de datos separado del nombre por un espacio. Por ejemplo... →

String no es un tipo primitivo. →

```
int numeroEntero;  
long numeroEnteroLargo;  
float numeroDecimal;  
double numeroDecimalDoblePrecision;  
boolean valorBooleano;  
String cadenaDeTexto;  
char unCaracter;
```

Variables - Valores literales

Al disponer Java de varios tipos de datos para representar los números y cadenas de caracteres, nos podemos encontrar con casos en los que un mismo valor pudiera ser válido para varios de los tipos de datos existentes.

- Cualquier valor que esté encerrado entre los caracteres **comillas dobles** será interpretado como una **cadena de caracteres**

```
int numero = "10";    // error, "10" es una cadena de texto
String numero = "10"; // correcto
```

- Cualquier valor que esté encerrado entre los caracteres **comillas simples** será interpretado como un valor de tipo **char**

```
int numero = '1';    // error, '1' es un caracter
char caracter = 'c'; // correcto
```


Variables - Valores literales

- Cualquier valor que NO esté encerrado entre ningún tipo de caracter será interpretado como un valor **numérico**.

```
String cadena = 10;    // error, 10 es un número  
int numero = 20;      // correcto
```

- Por defecto, cualquier número entero será interpretado como entero (int), aunque Java permite que sean asignados a variables de tipos más cortos (menor longitud en bits). Por ejemplo, si se quiere representar un valor literal de tipo (**long**) se puede acompañar con el carácter (l) a la derecha del valor.

```
int numero = 501;      // error, es un valor long  
long numeroLargo = 12345678901; // correcto  
long numeroLargo = 1234567890l; // también es correcto
```

- Por defecto, cualquier número de coma flotante será interpretado como de doble precisión (**double**).

```
float peso = 10.4;     // error, pérdida de precisión  
double peso = 10.4;    // correcto  
float altura = 1.80f;  // correcto
```

Variables - Valores por defecto y ámbito

- Todos los **valores numéricos** tomarán el valor **0** por defecto.
- Las **variables booleanas** tomarán el valor **false** por defecto.
- El **tipo char** tomará valor nulo (**null**) por defecto.
- Los tipos como **String** no tiene valor por defecto (**null**)

Ámbito de las variables

El alcance de las variables se refiere al ámbito en que éstas pueden ser utilizadas. Como regla general hay que considerar que **una variable podrá ser utilizada sólo dentro del bloque de código donde ha sido declarada**. Más adelante, a medida que avance el curso, iremos viendo diferentes conceptos de bloque de código o diferentes tipos de ámbito con los que podremos encontrarnos. Por ahora, vamos a suponer que **las variables las podremos utilizar a partir de donde han sido declaradas, y nunca antes de hacerlo**.

- Las **variables globales** son aquellas que se declaran al inicio del programa y perduran durante toda su ejecución. Pueden ser utilizadas desde cualquier parte del mismo.
- Las **variables locales** son aquellas que se declaran en algún bloque del programa y perduran solamente dentro de ese bloque. Podrán ser, por tanto, utilizadas solamente dentro del bloque donde fueron declaradas.

Variables - Conversión de tipos

Por norma general, siempre podremos convertir el valor de una variable a otra variable de distinto tipo, siempre y cuando tenga sentido representar dicho valor como otro tipo de dato.

Conversión implícita

Algunas conversiones resulta bastante triviales y las hará Java por nosotros.

```
int numero = 3;  
float otroNumero = numero;  
System.out.println(otroNumero);    // 3.0
```

```
float numero = 3.45f;  
int otroNumero = numero;    // error: incompatible types:  
System.out.println(otroNumero);
```

Variables - Conversión de tipos

Conversión Explícita

Para poder llevar a cabo por el momento una conversión explícita de tipos sencilla, podemos interponer el tipo de dato al que queramos convertir el valor (encerrado entre paréntesis). Aunque hay que tener en cuenta que esto sólo se podrá hacer entre tipos de datos compatibles.

```
float numero = 3.45f;  
int otroNumero = (int) numero;  
System.out.println(otroNumero);    // 3
```

```
float numero = 3.45f;  
String cadena = String.valueOf(numero);  
System.out.println(cadena);        // 3.45
```

```
String cadena = "1234";  
int numero = Integer.parseInt(cadena); // correcto  
String otraCadena = "hola";  
int otroNumero = Integer.parseInt(otraCadena); // error
```

```
String cadena = "1234";  
float numero = Float.parseFloat(cadena); // correcto  
String otraCadena = "1234f";  
float otroNumero = Float.parseFloat(otraCadena); // correcto
```

```
String cadena = "true";  
boolean valorBooleano = Boolean.parseBoolean(cadena); // correcto  
String otraCadena = "hola";  
boolean otroValorBooleano = Boolean.parseBoolean(otraCadena); // correcto
```

Constantes

Las constantes son estructuras que se definen con un valor fijo que no se puede modificar. Se utilizan para evitar tener que escribir números literales a lo largo del código, cuyos valores pueden cambiar en un futuro. De esa manera, colocaremos el nombre de la constante que defina dicho valor y en el momento en que ésta cambie, sólo tendremos que cambiar el valor en el lugar donde fue definida.

Hay que tener en cuenta que Java recomienda utilizar una notación diferente para los identificadores de las constantes. Se recomienda utilizar sólo mayúsculas y si el nombre contiene más de una palabra, se deben de separar con el carácter subrayado _

```
// Constantes para uso local
final float IVA = 0.21;
final int NUMERO_PAGINAS = 10;

// Constantes para uso global
public static final float IVA = 0.21;
public static final int NUMERO_PAGINAS = 10;
```


Convenciones de nombres de variables



camelCase



kebab-case



snake_case

Reglas para CamelCase

Clase	Métodos	Variables
Comienzan por mayúscula	Comienzan por minúscula y llevan llaves	Comienzan por minúscula
App LectorArchivos InterfazGrafica	sumarNumeros() importarTexto() dibujarFigura	radio resultadoSuma resolucionX

- Son case sensitive
- Cuando están formadas por varias palabras, las siguientes empiezan siempre por mayúscula

Operaciones básicas

Operaciones aritméticas

Símbolo	Descripción
+	Suma
+	Más unario: positivo
-	Resta
-	Menor unario: negativo
*	Multiplicación
/	División
%	Módulo
++	Incremento en 1
--	Decremento en 1

Operaciones básicas

Operaciones relacionales

Símbolo	Descripción
==	Igual que
!=	Distinto que
<	Menor que
<=	Menor o igual que
>	Mayor que
>=	Mayor o igual que

Operaciones básicas

Operaciones lógicas

Símbolo	Descripción
&&	Operador and: Y
	Operador or: O
!	Operador not: negación

Operaciones operación y asignación

Símbolo	Descripción
+=	Suma y asigna
-=	Resta y asigna
*=	Multiplica y asigna
/=	Divide y asigna
%=	Módulo y asigna

Api de Java - Clase scanner

La clase Scanner que permite leer una entrada de datos, por ejemplo, la entrada por teclado para almacenar en una variable lo que el usuario escribe.

```
Scanner keyboard = new Scanner(System.in);  
String input = keyboard.nextLine();  
String integerNumber = keyboard.nextInt();
```

- `sc.nextInt()`: lee un número entero (`int`) por teclado.
- `sc.nextDouble()`: lee un número real (`double`).
- `sc.nextLine()`: lee una cadena de caracteres (una frase) hasta que se pulsa Intro.
- `sc.next()`: lee una cadena de caracteres hasta que se encuentra un tabulador, un espacio en blanco o un Intro.

Cuidado

```
1 import java.util.Scanner;
```

Api de Java - Clases math y Random

La clase Math es una clase con métodos estáticos para realizar operaciones matemáticas

- `abs(numero)`: Valor absoluto de un número (útil para calcular la diferencia o distancia entre dos valores)
- `ceil(numero)`: Obtiene el siguiente valor entero por encima del que se pasa como parámetro
- `floor(numero)`: Obtiene el anterior valor entero por debajo del que se pasa como parámetro
- `max(numero1, numero2)`: Obtiene el máximo de dos valores dados
- `min(numero1, numero2)`: Obtiene el mínimo de dos valores dados
- `random()`: Devuelve un número aleatorio entre 0 y 1
- `round(numero)`: Redondea el número que se pasa como parámetro
- `sqrt(numero)`: Calcula la raíz cuadrada del número que se pasa como parámetro

Cuidado

1

```
import java.lang.Math.*;
```

La clase Random permite generar valores pseudoaleatorios algo más cómodos que los generados con **Math.random()** (números enteros, números enteros entre un rango dado, valores booleanos, ...)

```
Random generator = new Random();  
int integerNumber = generator.nextInt(10);  
boolean booleanValue = generator.nextBoolean();
```

Cuidado

1

```
import java.util.Random;
```

Sentencias básicas

Mostrar un mensaje en pantalla

```
System.out.println("Hola que tal");  
...  
int cantidad = 10;  
System.out.println("La cantidad es " + cantidad);
```

Generar un número aleatorio entre 0 y 10

```
Random generador = new Random();  
int numeroAleatorio = generador.nextInt(10);
```

Simular el lanzamiento de una moneda

```
Random generador = new Random();  
boolean lanzamientoMoneda = generador.nextBoolean();
```

Leer la entrada del usuario por teclado

```
Scanner teclado = new Scanner(System.in);  
String entrada = teclado.nextLine();
```